Onion De Bruijn Sequences: Fixed-Window Counting by Growing the Alphabet

Dor Genosar, Yotam Svoray, Gera Weiss

Abstract

We study De Bruijn sequences by fixing the word length n and enlarging the alphabet as needed. The reverse prefer-max sequence is compatible with this growth: for each k, its restriction to $[k]^n$ is a De Bruijn sequence, yielding an infinite sequence over $\mathbb{N}$. We formalize this through the notion of an onion De Bruijn sequence, prove the resulting structural properties, and count compatible finite onion prefixes by an explicit product formula. This leads to an arithmetic view in which integers are encoded by fixed-length words, with the maximal symbol indicating the layer. For orders $n = 2, 3$, we give explicit rank/unrank formulas and describe addition and multiplication via finite normalization, with exact carry counts and linear carry complexity in the input layers.

Contents

1 Introduction **1**

2 The Onion Theorem and Onion De Bruijn Sequences **3**
2.1 The Onion Theorem for Prefer-Max 3
2.2 Structure of onion De Bruijn sequences 5
2.3 Counting layer orders and onion prefixes 6

3 Arithmetic properties of onion representations **9**
3.1 Direct arithmetic in order 2 10
3.2 Direct arithmetic in order 3 16

4 Conclusions and future directions **22**

A Computational verification and software **25**

1 Introduction

Given positive integers n and k, a De Bruijn sequence of order n over the alphabet $[k] = \{0, \ldots, k-1\}$ is a cyclic sequence $(w_i)_{i=0}^{k^n-1}$ of words in $[k]^n$ such that

1. If $w_i = \sigma_1 \cdots \sigma_n$ for $\sigma_1, \ldots, \sigma_n \in [k]$ then $w_{i+1 \bmod k^n} = \sigma_2 \cdots \sigma_n \tau$ for some $\tau \in [k]$.

2. If $i \neq j$ then $w_i \neq w_j$.

For example, for $n = 2$ and $k = 2$, the sequence $00, 01, 11, 10$ is a De Bruijn sequence. One can view De Bruijn sequences as Hamiltonian cycles in the De Bruijn digraph $DB(n, k)$, whose vertices are elements of $[k]^n$ and where $\sigma_1 \cdots \sigma_n$ is connected to $\tau_1 \cdots \tau_n$ if and only if $\sigma_i = \tau_{i+1}$ for every i. De Bruijn sequences are named after N. G. de Bruijn, who studied them systematically in [6].

Most work on infinite or extendable De Bruijn sequences varies the order parameter n while keeping the alphabet fixed. A representative result in this direction is the following theorem of Becher and Heiber [5]:

Theorem (Theorem 1 in [5])**.** *Every De Bruijn sequence of order n in at least three symbols can be extended to a De Bruijn sequence of order $n+1$ (over the same alphabet). Every De Bruijn sequence of order n in two symbols cannot be extended to order $n+1$, but it can be extended to order $n+2$.*

A complementary question is to keep the order n fixed and to grow the alphabet instead. Can a De Bruijn sequence of order n over $[k]$ be extended to one over $[k+1]$ without changing the window[1] length? Equivalently, can one coherently enumerate the sets $[k]^n$ for all k so that the k-th stage extends the previous one? This leads to the following definition.

Definition 1. *We say that a sequence $(x_i)_{i=0}^{\infty}$ of words in $\mathbb{N}^n$ is* ***an onion De Bruijn sequence*** *of order n if, for every $k \geq 1$, the prefix $(x_i)_{i=0}^{k^n-1}$ is a De Bruijn sequence of order n over the alphabet $[k]$.*

An onion De Bruijn sequence is therefore a coherent enumeration of the sets $[k]^n$ as the alphabet grows. The name reflects the resulting layer structure. It may be viewed as a fixed-window counting system: the register length stays equal to n, and larger integers are represented by enlarging the alphabet only when necessary. For this interpretation to be algorithmically useful, the successor should be computable locally from the current window, i.e. by a shift rule; see, for example, [3]. In the onion setting such a rule would keep the counter running while the alphabet grows.

Our first main result, Theorem 3, shows that the reverse prefer-max De Bruijn sequence has the onion property, yielding a canonical infinite sequence over $\mathbb{N}$. We then identify the structural features shared by every onion sequence, count the compatible layer orders and finite onion prefixes, and translate the layer decomposition into arithmetic consequences. For the reverse prefer-max onion sequences of orders 2 and 3, these consequences become explicit rank/unrank formulas together with direct addition and multiplication by finite layer carries, including exact carry counts and linear carry complexity.

[1] By "window" we mean as an element of the total De Bruijn sequence, i.e. a word of length n.

The paper follows this progression. Theorem 3 proves the onion property for reverse prefer-max. Section 2.2 isolates the universal layer constraints, and Section 2.3 counts layer orders and compatible finite onion prefixes. The next three sections develop the arithmetic side: first in general, then explicitly in orders 2 and 3. The final section summarizes the picture and the main open directions.

Notation. Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. For a word $y = \sigma_1 \cdots \sigma_m \in [k]^*$, let $\mu_y = \max_i \sigma_i$. We write $\tau \in w$ if the symbol τ occurs in the word w. For w and $m < r \leq n$, let $w[m..r]$ denote the suffix of w of length $r - m + 1$.

Acknowledgments: We wish to thank Ben Keefer for his input on some of the ideas in Section 2.2.

2 The Onion Theorem and Onion De Bruijn Sequences

2.1 The Onion Theorem for Prefer-Max

Definition 2. *The (k, n)-**prefer-max De Bruijn sequence**, denoted by $(w_i)_{i=0}^{k^n-1}$, is defined recursively as follows:*

- $w_0 = 0^{n-1}(k-1)$,
- *If $w_i = \sigma x$ for some $\sigma \in [k]$ and $x \in [k]^{n-1}$ then $w_{i+1} = x\tau$ where τ is the maximal*[2] *symbol in $[k]$ such that $w_j \neq x\tau$ for all $j \leq i$.*

The prefer-max De Bruijn sequence was first defined and studied in the binary case in [13, 8]. Our first result identifies a suffix relation between the (k, n)-prefer-max and the $(k-1, n)$-prefer-max sequences:

Theorem 3 (Onion Theorem)**.** *For every $k > 1$ and $n \geq 2$, if $(w_i)_{i=0}^{k^n-1}$ is the (k, n)-prefer-max sequence, then its suffix $(w_i)_{i=k^n-(k-1)^n}^{k^n-1}$ is the $(k-1, n)$-prefer-max De Bruijn sequence.*

Proof. The case $n = 2$ can be read off directly from the prefer-max rule. For $n > 2$, the proof has three steps. First, we locate the first word that avoids the symbol $k - 1$ and show that the preceding transition occurs immediately after the word $(k-1)0^{n-1}$. Second, we prove that every word containing $k - 1$ appears before that point. Once these two facts are established, the remaining suffix uses only the alphabet $[k-1]$, and the prefer-max rule continues exactly as it does for the $(k-1, n)$-sequence.

For the argument below, we view the prefer-max cycle through an underlying linearized symbol string (σ_i) whose consecutive length-n windows are the words w_i. This lets us refer to the unique symbol that exits or enters when one moves from one window to the next.

[2] We view $[k]$ with the usual order, i.e. $0 < 1 < \cdots < k - 1$.

We first treat the case $n = 2$. By the prefer-max rule, the initial block of words containing the symbol $k-1$ is

$$0(k-1),\ (k-1)(k-1),\ (k-1)(k-2),\ (k-2)(k-1),\ \ldots,$$
$$(k-1)1,\ 1(k-1),\ (k-1)0.$$

Indeed, after the word $j(k-1)$ with $1 \le j \le k-1$, the next word must have prefix $k-1$, and the largest unseen choice is $(k-1)(j-1)$. Likewise, after the word $(k-1)j$ with $1 \le j \le k-2$, the next word must have prefix j, and the largest unseen choice is $j(k-1)$. After $(k-1)0$, the next word is $0(k-2)$, and from that point on all words containing $k-1$ have already appeared. Hence the construction proceeds exactly as the prefer-max construction over the alphabet $[k-1]$, so the remaining suffix is the $(k-1,2)$-prefer-max sequence.

Assume from now on that $n > 2$. Step 1 is to identify the first transition away from the symbol $k-1$. We show that all words in $[k]^n \setminus [k-1]^n$ occur before the first word that avoids the symbol $k-1$. Let $i_0 = \min\{i \colon k-1 \notin w_i\}$. By minimality, since σ_{i_0-n} is the only symbol in $[k]$ such that $\sigma_{i_0-n} \in w_{i_0-1}$ but $\sigma_{i_0-n} \notin w_{i_0}$, we have that $\sigma_{i_0-n} = k-1$. Since $\sigma_{i_0} \neq k-1$ and $\sigma_{i_0-n+1} \cdots \sigma_{i_0-1}(k-2) \notin \{w_1, \ldots, w_{i_0-1}\}$ (as it does not contain the symbol $k-1$), we get from the construction of the prefer-max sequence that $\sigma_{i_0} = k-2$. This means that there exists some $i_1 < i_0$ such that $w_{i_1} = \sigma_{i_0-n+1} \cdots \sigma_{i_0-1}(k-1)$.

Assume, towards contradiction, that $\sigma_{i_0-n+1} \cdots \sigma_{i_0-1} \neq 0^{n-1}$. In particular $w_{i_1} \neq 0^{n-1}(k-1)$ which means that $i_1 > 1$. Therefore, $k-1 \in w_{i_1-1}$ and thus $k-1 \in \{\sigma_{i_1-n}, \ldots, \sigma_{i_1-1}\} = \{\sigma_{i_1-n}\} \cup \{\sigma_{i_0-n+1}, \ldots, \sigma_{i_0-1}\}$. Since the second set in this union does not contain $k-1$, we get that $\sigma_{i_1-n} = k-1$. This leads to a contradiction because it means that $w_{i_1-1} = w_{i_0-1}$ which cannot happen in a De Bruijn sequence. This contradiction gives us that $\sigma_{i_0-n+1} \cdots \sigma_{i_0-1} = 0^{n-1}$. The sequence is therefore as follows:

$$\underbrace{\sigma_1, \ldots, \sigma_{i_0-n-1}, k-1, \overbrace{0, \ldots, 0}^{n\text{-1 times}}}_{\text{all the } w_i\text{s here contain } k-1}, \underbrace{k-2}_{\sigma_{i_0}}, \sigma_{i_0+1}, \ldots, \sigma_{k^n+1}$$

Step 2 is to show that all substrings of length n that contain $k-1$ appear before σ_{i_0}.

Given some $v \in [k]^n$ and $1 \le m \le n$, let j_m be the index such that w_{j_m} is of the form $v[m..n]0^{m-1}$. Because zero is the smallest symbol in our alphabet, we get that it is added by the prefer-max rule only if all the other symbols cannot be added, i.e., when adding other symbols would generate a substring that has already been seen before. Since v comes, as a window, right before a word of the form $v[2..n]\tau$ for some $\tau \ge 0$, we have that $v[2..n]0$ comes after (or is equal to) $v[2..n]\tau$ which is after v. Similar arguments give us that $v[2..n]0$ appears before $v[3..n]0^2$ which appears before $v[4..n]0^3$ and so on. Together we get that $j_1 < j_2 < \cdots < j_n$. Note that $w_{j_1} = v$, i.e., that v appears as a window at the j_1-th symbol of the sequence.

We now show that if v contains the symbol $k-1$, then it must appear as a substring before the i_0-th symbol in the sequence, i.e., $j_1 < i_0$. Consider first the case where the last (right-most) symbol of v is $k-1$. In this case, $j_n = i_0 - 1$ so we have, because $j_1 < j_n$, that v appears as a substring before the i_0-th symbol in the sequence. The second case is when the last (right-most) symbol in v is not $k-1$ but there is another letter in v which is $k-1$. In this case we have that v appears less than n steps after a window that ends with $k-1$ and the window $w_{i_0-1} = (k-1)0^{n-1}$ does not appear in the windows between them because they all contain $k-1$ at a letter that is not their first. Formally, we have that there is some $m < n$ such that $j_1 - m < i_0$, $v \in \{w_{j_1-m}, \ldots, w_{j_1-1}\}$ and $w_{i_0-1} \notin \{w_{j_1-m}, \ldots, w_{j_1-1}\}$. This gives us that, also in this case, the window w appears before the index i_0.

Step 3 is now immediate: we established that the windows before the index i_0 contain all and only the words in $[k]^n \setminus [k-1]^n$ and that this part of the sequence ends with $n-1$ zeros and then the next part begins with the symbol $k-2$. This means that from index i_0 onward the prefer-max construction acts exactly as it does over the alphabet $[k-1]$, so $(w_i)_{i=i_0}^{k^n-1}$ is the $(k-1,n)$-prefer-max sequence. Since

$$i_0 = |[k]^n \setminus [k-1]^n| = k^n - (k-1)^n,$$

this is precisely the claimed suffix. □

Example 4. The $(2,2)$-prefer-max sequence is 01100, and the $(3,2)$-prefer-max sequence is 0221201100; the former appears as a suffix of the latter.

By Theorem 3, for every $n \geq 2$ the reverse of the $(2,n)$-prefer-max sequence is a prefix of the reverse of the $(3,n)$-prefer-max sequence, which is a prefix of the reverse of the $(4,n)$-prefer-max sequence, and so on. Thus the reversed prefer-max sequences define an infinite onion De Bruijn sequence of order n, i.e., an infinite sequence in which every word in $\mathbb{N}^n$ appears exactly once as a contiguous block.

This theorem also clarifies why shift-register constructions are so natural in the onion setting: compatibility across alphabet sizes becomes a local successor problem. One wants a rule that updates the current window by shifting and appending one symbol, while still remaining coherent as the alphabet grows.

Remark 5. An alternative proof of Theorem 3 was presented in Theorem 26 of [4] using the cycle-joining construction presented in that paper.

2.2 Structure of onion De Bruijn sequences

By Theorem 3, the reverse prefer-max construction gives one onion De Bruijn sequence. This section records the structural features shared by all of them. First we show that, once a context of length $n-1$ is fixed, enlarging a symbol beyond the current maximum preserves the order. Then we identify the resulting layer decomposition by maximal symbol.

Proposition 6. *Let $(x_i)_{i=0}^{\infty}$ be an onion De Bruijn sequence of order n. Then for every word $y \in \mathbb{N}^{n-1}$ the sequence $\{y\tau \colon \tau \geq \mu_y\}$ appears as an increasing subsequence of $(x_i)_{i=0}^{\infty}$.*

Proof. Since $(x_i)_{i=0}^{\infty}$ is an onion De Bruijn sequence, the set $\{y\sigma \colon \sigma \leq \mu_y\}$ already appears in the $(\mu_y + 1, n)$-De Bruijn prefix of $(x_i)_{i=0}^{\infty}$. Assume towards contradiction that there exist $y \in [k]^{n-1}$ for some positive integer k and symbols $\mu_y < \sigma_1 < \sigma_2$ such that $x_{i_1} = y\sigma_2$ and $x_{i_2} = y\sigma_1$ with $i_1 < i_2$. Since $y\sigma_1 \in [\sigma_1 + 1]^n$, the word $y\sigma_1$ appears in the prefix $(x_i)_{i=0}^{(\sigma_1+1)^n-1}$. On the other hand, $y\sigma_2 \notin [\sigma_1 + 1]^n$, so its unique occurrence cannot lie in that prefix. Hence $i_2 \leq (\sigma_1 + 1)^n - 1 < i_1$, a contradiction. □

The following corollary is an analogue of Lemma 3 in [3] for onion De Bruijn sequences:

Corollary 7. *Let $(x_i)_{i=0}^{\infty}$ be an onion De Bruijn sequence of order n, and let w and u be two words whose lengths sum to $n-1$. Then the subsequence $\{w\sigma u \colon \sigma > \mu_{wu}\}$ appears in increasing order in $(x_i)_{i=0}^{\infty}$.*

Proof. The proof is the same as that of Proposition 6, except that one fixes the context w and u around the varying symbol instead of fixing only a prefix. □

Proposition 8. *Let $(x_i)_{i=0}^{\infty}$ be an onion De Bruijn sequence of order n. Then for every $k \geq 2$ and every index $i \geq 0$,*

$$x_i \in [k]^n \setminus [k-1]^n \quad \Longleftrightarrow \quad (k-1)^n \leq i \leq k^n - 1.$$

Proof. Let $k \geq 2$. First, assume that $(k-1)^n \leq i \leq k^n - 1$. Since $(x_i)_{i=0}^{\infty}$ is an onion De Bruijn sequence of order n, the prefix $(x_i)_{i=0}^{(k-1)^n-1}$ is an $(n, k-1)$-De Bruijn sequence and the prefix $(x_i)_{i=0}^{k^n-1}$ is an (n, k)-De Bruijn sequence. Thus every word in $[k-1]^n$ already appears exactly once among $x_0, \ldots, x_{(k-1)^n-1}$. Since the longer prefix is a De Bruijn sequence over $[k]$, we also have $x_i \in [k]^n$. Because $i \geq (k-1)^n$, the word x_i cannot belong to $[k-1]^n$, and therefore $x_i \in [k]^n \setminus [k-1]^n$.

Conversely, assume that $x_i \in [k]^n \setminus [k-1]^n$. The prefix $(x_i)_{i=0}^{k^n-1}$ is an (n, k)-De Bruijn sequence, so every word in $[k]^n$ appears exactly once among $x_0, \ldots, x_{k^n-1}$. Hence $i \leq k^n - 1$. On the other hand, the prefix $(x_i)_{i=0}^{(k-1)^n-1}$ already contains every word in $[k-1]^n$ exactly once, and $x_i \notin [k-1]^n$. Therefore $i \geq (k-1)^n$, and so $(k-1)^n \leq i \leq k^n - 1$. □

2.3 Counting layer orders and onion prefixes

The layer decomposition also has an enumerative side. For fixed n and k, let $L_{n,k}$ denote the induced subgraph of the De Bruijn digraph $DB(n, k)$ on the vertex set $[k]^n \setminus [k-1]^n$. A Hamiltonian cycle in $L_{n,k}$ is exactly an ordering of the k-th layer that respects the De Bruijn overlap condition.

Theorem 9 (Enumeration of the k-th layer)**.** *For every $n \geq 1$ and $k \geq 1$, the number of Hamiltonian cycles in $L_{n,k}$ is*

$$\frac{k!^{\,k^{n-1}-(k-1)^{n-1}}}{k^{n-1}}.$$

Proof. The case $n = 1$ is immediate, since $L_{1,k}$ consists of the single vertex $k-1$. Assume therefore that $n \geq 2$, and write $m = n-1$. Consider the digraph $G_{n,k}$ whose vertex set is $[k]^m$, and in which $x\tau \longrightarrow \sigma x$ is an edge whenever $\sigma x\tau \in [k]^n \setminus [k-1]^n$. The edges of $G_{n,k}$ are naturally indexed by the vertices of $L_{n,k}$ and two edges $x\tau \to \sigma x$ and $y\eta \to \varphi y$ are consecutive if and only if $\sigma x = y\eta$, which is exactly the condition that the corresponding layer words $\sigma x\tau$ and $\varphi y\eta$ overlap in $n-1$ symbols. Hence the line digraph of $G_{n,k}$ is precisely $L_{n,k}$ and therefore Hamiltonian cycles in $L_{n,k}$ are in bijection with Eulerian circuits in $G_{n,k}$.

Note that the digraph $G_{n,k}$ is Eulerian since if a vertex $u \in [k]^m$ contains the symbol $k-1$, then any symbol of $[k]$ may be prepended or appended while staying inside $[k]^n \setminus [k-1]^n$, so $\deg_{\text{in}}(u) = \deg_{\text{out}}(u) = k$, and if $u \in [k-1]^m$, then only the symbol $k-1$ may be prepended or appended, and therefore $\deg_{\text{in}}(u) = \deg_{\text{out}}(u) = 1$. In addition, the digraph is also strongly connected, i.e. from any vertex one reaches $(k-1)^m$ by repeatedly prepending $k-1$, and from $(k-1)^m$ one reaches any prescribed vertex by prepending its symbols from right to left.

Denote $S = [k-1]^m$ and $L = [k]^m \setminus [k-1]^m$ and let $\mathcal{L}$ be the out-degree Laplacian of $G_{n,k}$. With respect to the decomposition $[k]^m = S \sqcup L$, the Laplacian has the block form

$$\mathcal{L} = \begin{pmatrix} I_{|S|} & -A_{SL} \\ -A_{LS} & kI_{|L|} - A_{LL} \end{pmatrix},$$

where A_{SL}, A_{LS}, A_{LL} are the corresponding adjacency blocks. Taking the Schur complement of the S-block in $\lambda I - \mathcal{L}$ gives us (after simplification) that the characteristic polynomial of $\mathcal{L}$ is

$$\chi_{\mathcal{L}}(\lambda) = \lambda(\lambda-1)^{|S|-1}(\lambda-k)^{|L|},$$

and therefore the nonzero Laplacian eigenvalues are 1 with multiplicity $|S|-1$ and k with multiplicity $|L|$. Now, fixing the vertex $r = (k-1)^m$, since $G_{n,k}$ is strongly connected and Eulerian, the directed Matrix-Tree theorem (see, for example, [15, 12]) tells us that the number of arborescences in $G_{n,k}$ rooted at r is given by

$$t_r(G_{n,k}) = \frac{1}{|[k]^m|} \prod_{\mu \neq 0} \mu = \frac{1}{k^m} 1^{|S|-1} k^{|L|} = k^{|L|-m},$$

where the product runs over all nonzero eigenvalues of $\mathcal{L}$. Since $|L| = k^m - (k-1)^m$ we can conclude that $t_r(G_{n,k}) = k^{k^m-(k-1)^m-m}$. Finally, by the BEST

theorem [16, 17], the number of Eulerian circuits in $G_{n,k}$ is

$$t_r(G_{n,k}) \prod_{u \in [k]^m} (\deg_{\text{out}}(u) - 1)!.$$

Since $(\deg_{\text{out}}(u) - 1)! = 0! = 1$ for every $u \in S$ and $(\deg_{\text{out}}(u) - 1)! = (k-1)!$ for every $u \in L$, we get that the number of Hamiltonian cycles in $L_{n,k}$ is exactly

$$k^{k^m - (k-1)^m - m}(k-1)!^{\,k^m - (k-1)^m} = \frac{k!^{\,k^{n-1} - (k-1)^{n-1}}}{k^{n-1}},$$

as desired. □

Example 10. In the case $n = 2$, Theorem 9 tells us that the number of Hamiltonian cycles in $L_{2,k}$ is $(k-1)!$, and we can see this directly as a Hamiltonian cycle in $L_{n,k}$ (without loss of generality) must be of the form

$$0(k-1) \to (k-1)j_1 \to j_1(k-1) \to (k-1)j_2 \to \cdots \to j_{k-1}(k-1) \to (k-1)0,$$

where $j_1, \ldots, j_{k-1}$ is any re-ordering of $1, \ldots, (k-1)$. In particular, the reverse of the prefer-max sequence is the case where $j_i = i$, as we see in Theorem 3.

Corollary 11 (Enumeration of compatible onion prefixes)**.** *For every $n \geq 1$ and $j \geq 1$, the number of finite sequences $(x_i)_{i=0}^{j^n-1}$ such that, for each $1 \leq k \leq j$, the prefix $(x_i)_{i=0}^{k^n-1}$ is a De Bruijn sequence of order n over the alphabet $[k]$ is*

$$\prod_{k=2}^{j} \frac{k!^{\,k^{n-1} - (k-1)^{n-1}}}{k^{n-1}},$$

where the empty product is interpreted as 1.

Proof. The case $j = 1$ is trivial, since the only such sequence is (0^n). Assume $j \geq 2$.

Fix $2 \leq k \leq j$. Since the prefix of length $(k-1)^n$ already contains every word of $[k-1]^n$ exactly once, while the prefix of length k^n contains every word of $[k]^n$ exactly once, the block

$$x_{(k-1)^n}, x_{(k-1)^n+1}, \ldots, x_{k^n-1}$$

consists exactly of the words in $[k]^n \setminus [k-1]^n$.

The last word of the length-k^n prefix must overlap the first word $x_0 = 0^n$ in the cyclic De Bruijn order, so it has the form $\tau 0^{n-1}$. Because it lies in $[k]^n \setminus [k-1]^n$, we must have that $\tau = k-1$, and so $x_{k^n-1} = (k-1)0^{n-1}$. Similarly, the first word of the k-th layer must overlap the last word of the $(k-1)$-st prefix, which is $(k-2)0^{n-1}$, and must lie in $[k]^n \setminus [k-1]^n$, and therefore $x_{(k-1)^n} = 0^{n-1}(k-1)$. Consequently the k-th layer is a Hamiltonian path in $L_{n,k}$ from $0^{n-1}(k-1)$ to $(k-1)0^{n-1}$.

These two endpoints are adjacent in $L_{n,k}$, because both belong to $[k]^n \setminus [k-1]^n$ and $(k-1)0^{n-1} \to 0^{n-1}(k-1)$ is a valid De Bruijn overlap. Hence Hamiltonian paths in $L_{n,k}$ from $0^{n-1}(k-1)$ to $(k-1)0^{n-1}$ are in bijection with Hamiltonian cycles in $L_{n,k}$. By Theorem 9, there are exactly $\frac{k!\,k^{n-1}-(k-1)^{n-1}}{k^{n-1}}$ choices for the k-th layer.

The choices for different values of k are independent, because the boundary words between consecutive layers are forced. Concatenating the chosen layers therefore produces exactly one compatible prefix of length j^n, and every such prefix arises uniquely in this way. Multiplying over $k = 2, \ldots, j$ gives the formula. □

Theorem 9 and Corollary 11 show that onion sequences are far from unique: each layer admits many Hamiltonian orders, and these choices are independent once the boundary words are fixed. Reverse prefer-max is therefore distinguished not by uniqueness, but by the extra arithmetic regularity of one particular compatible choice.

3 Arithmetic properties of onion representations

Let $(x_i)_{i=0}^{\infty}$ be an onion De Bruijn sequence of order n. Since every word in $\mathbb{N}^n$ appears exactly once, we may define its **rank** by $\rho(w) = i$ if and only if $x_i = w$. This turns the onion sequence into a representation of the nonnegative integers by words of fixed length n.

The first observation is that the maximal symbol determines the represented magnitude exactly: the k-th layer occupies the interval between two consecutive perfect nth powers.

Proposition 12. *For every word $w \in \mathbb{N}^n$ we have $\mu_w^n \leq \rho(w) < (\mu_w + 1)^n$. Equivalently, $\mu_w = \lfloor \rho(w)^{1/n} \rfloor$.*

Proof. Let $k = \mu_w + 1$. Then $w \in [k]^n \setminus [k-1]^n$. By Proposition 8, the words in $[k]^n \setminus [k-1]^n$ are precisely the words appearing in the interval of indices $[(k-1)^n, k^n - 1]$. Therefore $(k-1)^n \leq \rho(w) \leq k^n - 1$. Substituting $k = \mu_w + 1$ gives the claim. □

Thus the maximal symbol plays the role of a most-significant digit, except that the thresholds are the perfect nth powers $0^n, 1^n, 2^n, \ldots$ because the word length is fixed and the alphabet is what grows.

Corollary 13 (Layer carries)**.** *Let $w \in \mathbb{N}^n$ and let $m \in \mathbb{N}$. If $z = \rho^{-1}(\rho(w)+m)$, then μ_z is the unique integer t satisfying*

$$t^n \leq \rho(w) + m < (t+1)^n.$$

In particular, under the successor map the maximal symbol changes precisely when one crosses a perfect nth-power boundary.

Proof. This is an immediate reformulation of Proposition 12. □

Corollary 14 (Dominant-digit monotonicity)**.** *Let w and u be two words whose lengths sum to $n-1$. Then for every $\sigma > \mu_{wu}$ we have*

$$\sigma^n \leq \rho(w\sigma u) < (\sigma+1)^n,$$

and the map $\sigma \mapsto \rho(w\sigma u)$ is strictly increasing on $\{\sigma \in \mathbb{N} \colon \sigma > \mu_{wu}\}$.

Proof. Since $\sigma > \mu_{wu}$, we have $\mu_{w\sigma u} = \sigma$, so the interval bound follows from Proposition 12. The monotonicity statement follows from Corollary 7. □

Together, Proposition 12, Corollary 13, and Corollary 14 isolate two arithmetic features common to every onion De Bruijn sequence: carries between layers occur at predictable indices, and once one symbol dominates the rest of the word, increasing it moves forward in the represented order. The next two examples illustrate these points before we specialize to the reverse prefer-max sequence of order 2.

Example 15. To see the layer boundary concretely, consider the onion De Bruijn sequence of order 2 obtained from the reverse prefer-max construction. Its first terms are

$$00, 01, 11, 10, 02, 21, 12, 22, 20, 03, 31, 13, 32, 23, 33, 30, \ldots$$

Thus

$$\rho(12) = 6, \qquad \rho(20) = 8, \qquad \rho(03) = 9, \qquad \rho(31) = 10.$$

In particular, $\mu_{12} = 2$ and $2^2 \leq 6 < 3^2$, while $\mu_{31} = 3$ and $3^2 \leq 10 < 4^2$, illustrating Proposition 12. The transition from 20 to 03 crosses the boundary $3^2 = 9$, so this is exactly a layer carry in the sense of Corollary 13.

Example 16. To isolate dominant-digit monotonicity, keep the surrounding context fixed and vary only the symbol that exceeds it. In the same sequence,

$$\rho(10) = 3 < \rho(20) = 8 < \rho(30) = 15 < \rho(40) = 24$$

and also

$$\rho(12) = 6 < \rho(13) = 11 < \rho(14) = 18.$$

The first chain corresponds to the context $w = \varepsilon$ and $u = 0$, while the second corresponds to $w = 1$ and $u = \varepsilon$.

3.1 Direct arithmetic in order 2

The order-2 case already exposes a nontrivial carry mechanism. Inside layer k, the new symbol k alternates between the right and left positions until the layer is exhausted, and only then does the sequence move to layer $k+1$. Thus, for order 2, one can describe not only *when* a layer carry occurs, but also the entire intra-layer motion leading to it. This is the first place where the abstract onion structure becomes an explicit dynamical rule. The next proposition makes this precise by identifying the complete order of the k-th layer and the terminal transition that produces the carry to the next layer.

Proposition 17 (Explicit layer order for the reverse prefer-max sequence of order 2)**.** *Let $(x_i)_{i=0}^{\infty}$ be the onion De Bruijn sequence of order 2 obtained from the reverse prefer-max construction, and let ρ be its rank map. Then for every $k \geq 1$, the words of layer k appear in the order*

$$0k,\ k1,\ 1k,\ k2,\ 2k,\ \ldots,\ k(k-1),\ (k-1)k,\ kk,\ k0.$$

Equivalently,

$$\rho(jk) = k^2 + 2j \qquad (0 \leq j < k)$$

and

$$\rho(kj) = k^2 + 2j - 1 \qquad (1 \leq j \leq k).$$

Moreover,

$$\rho(k0) = k^2 + 2k = (k+1)^2 - 1.$$

Proof. Consider the forward prefer-max De Bruijn sequence of order 2 over the alphabet $[k+1]$. We claim that its initial block of words containing the symbol k is

$$0k,\ kk,\ k(k-1),\ (k-1)k,\ k(k-2),\ (k-2)k,\ \ldots,\ k1,\ 1k,\ k0.$$

Indeed, the first word is $0k$ by definition. Since the current suffix is then k, the prefer-max rule chooses kk next. More generally, after the word jk with $1 \leq j \leq k$, the next word must have prefix k, and among the candidates with that prefix the largest unseen one is $k(j-1)$. Likewise, after the word kj with $1 \leq j \leq k-1$, the next word must have prefix j, and the largest unseen choice is jk. This determines the whole block.

By the Onion Theorem, the reverse prefer-max onion sequence is obtained by reversing these layers. For words of length 2, reversing a word simply exchanges its two symbols, so the corresponding layer in the reverse prefer-max onion sequence is

$$0k,\ k1,\ 1k,\ k2,\ 2k,\ \ldots,\ k(k-1),\ (k-1)k,\ kk,\ k0.$$

Since layer k begins at index k^2, the displayed rank formulas follows. □

Proposition 17 is relevant because it turns the vague statement "carries propagate across layers" into a completely explicit successor rule for order 2. Instead of merely knowing the boundary between one layer and the next, we know the exact path followed inside each layer. Indeed, inside layer k the successor map alternates between the two positions:

$$jk \mapsto k(j+1) \quad (0 \leq j < k), \qquad kj \mapsto jk \quad (1 \leq j < k),$$

followed by the two terminal transitions

$$kk \mapsto k0, \qquad k0 \mapsto 0(k+1).$$

Thus the actual carry to the next alphabet layer happens only at the final step $k0 \mapsto 0(k+1)$.

Example 18. The new issue addressed by Proposition 17 is not where a layer begins, but how the successor moves *inside* that layer. For example, when $k = 5$ the rule gives

$$05 \to 51 \to 15 \to 52 \to 25 \to 53 \to 35 \to 54 \to 45 \to 55 \to 50 \to 06.$$

Thus the words alternate between ending in 5 and beginning with 5 until the terminal pair $55, 50$. Every step before $50 \to 06$ stays inside layer 5, and the only actual carry to the next layer is the final transition $50 \to 06$.

The previous proposition gives more than a description of the successor map: it yields an explicit arithmetic for the order-2 reverse prefer-max onion sequence. Once the rank map and its inverse are known in closed form, addition and multiplication can be transported from the ordinary arithmetic of integers to the onion representation. This is not yet a local digit-by-digit algorithm of the usual decimal type, but it is already a complete and exact arithmetic on the represented words. The resulting formulas are easy to parse conceptually: the square m^2 identifies the layer, while the offset from m^2 tells us where we are inside that layer.

Corollary 19 (Explicit rank and unrank for order 2)**.** *Let ρ_2 be the rank map of the reverse prefer-max onion sequence of order 2. Then for every $a, b \in \mathbb{N}$ we have*

$$\rho_2(a,b) = \begin{cases} b^2 + 2a, & a < b, \\ a^2 + 2b - 1, & 0 < b \le a, \\ a^2 + 2a, & b = 0. \end{cases}$$

Conversely, if $N \in \mathbb{N}$, $m = \lfloor \sqrt{N} \rfloor$, and $t = N - m^2$, then

$$\rho_2^{-1}(N) = \begin{cases} \left(\frac{t}{2}, m\right), & t \text{ is even and } t < 2m, \\ \left(m, \frac{t+1}{2}\right), & t \text{ is odd}, \\ (m, 0), & t = 2m. \end{cases}$$

Consequently, the operations

$$(a,b) \oplus_2 (c,d) = \rho_2^{-1}\big(\rho_2(a,b) + \rho_2(c,d)\big)$$

and

$$(a,b) \otimes_2 (c,d) = \rho_2^{-1}\big(\rho_2(a,b)\rho_2(c,d)\big)$$

are explicit addition and multiplication rules on the order-2 onion representations.

Proof. The formula for ρ_2 is a restatement of Proposition 17. The inverse formula follows by writing $N = m^2 + t$ with $0 \le t \le 2m$ and reading off the unique word in layer m having offset t in the list from Proposition 17. The formulas for $\oplus_2$ and $\otimes_2$ are then immediate. □

Corollary 19 transports ordinary addition and multiplication to the order-2 onion representation through the rank map. We now rewrite those operations directly in layer-offset coordinates.

By Proposition 17, the m-th layer is a zig-zag path

$$0m,\ m1,\ 1m,\ m2,\ 2m,\ \dots,\ m(m-1),\ (m-1)m,\ mm,\ m0.$$

Accordingly, write

$$\lambda_2(a,b) = \begin{cases} (b,a,0), & a < b, \\ (a,a,0), & b = 0, \\ (a,b,1), & 0 < b \le a. \end{cases}$$

If $\lambda_2(x) = (m,u,\varepsilon)$, then $\rho_2(x) = m^2 + 2u - \varepsilon$, where $\varepsilon \le u \le m$ and $\varepsilon \in \{0,1\}$. Thus m records the layer, u the offset inside the layer, and ε which branch of the zig-zag one is on.

Proposition 20 (Direct addition for order 2)**.** *Let $\lambda_2(x) = (m,u,\varepsilon)$ and $\lambda_2(y) = (n,v,\eta)$. Define the raw sum triple by*

$$M = m + n, \qquad U = u + v - mn - \left\lfloor \frac{\varepsilon+\eta}{2} \right\rfloor, \qquad E = (\varepsilon+\eta) \bmod 2.$$

Then

$$\rho_2(x) + \rho_2(y) = M^2 + 2U - E.$$

In the special case $\lambda_2(x) = (b,a,0)$ and $\lambda_2(y) = (d,c,0)$, this reduces to the identity $M = b+d$, $U = a+c-bd$, $E = 0$. Moreover $U \le M$, so the raw triple can fail to be canonical only on the low side. In that case one repeatedly applies the downward carry rule

$$(m,u,\varepsilon) \longmapsto (m-1, u+m-\varepsilon, 1-\varepsilon) \qquad \text{if } u+1-\varepsilon \le 0,$$

This process terminates at a unique canonical triple (m,u,ε), and the corresponding sum word is

$$x \oplus_2 y = \begin{cases} (u,m), & \varepsilon = 0 \text{ and } u < m, \\ (m,0), & \varepsilon = 0 \text{ and } u = m, \\ (m,u), & \varepsilon = 1. \end{cases}$$

Hence order-2 addition can be carried out directly in the onion representation, without global rank inversion.

Proof. Adding the identities $\rho_2(x) = m^2 + 2u - \varepsilon$ and $\rho_2(y) = n^2 + 2v - \eta$ gives

$$\rho_2(x)+\rho_2(y) = m^2+n^2+2u+2v-\varepsilon-\eta = (m+n)^2+2\Big(u+v-mn-\left\lfloor \frac{\varepsilon+\eta}{2} \right\rfloor\Big)-E,$$

which is the claimed raw-sum formula. Since $u \le m$ and $v \le n$, we have

$$U \le m + n - mn \le m + n = M,$$

so the raw triple cannot overshoot the top of layer M.

The two carry rules are the identities

$$m^2 + 2u = (m-1)^2 + 2(u+m) - 1$$

and

$$m^2 + 2u - 1 = (m-1)^2 + 2(u+m-1),$$

so each carry preserves the represented integer. If $u < 0$ in the even case then $m^2 + 2u < m^2$, while if $u \leq 0$ in the odd case then $m^2 + 2u - 1 < m^2$. In either situation, the represented integer lies strictly below layer m, so the canonical representative must occur in a smaller layer. Each carry lowers the layer by 1, hence the process must terminate. The final triple is unique because ρ_2^{-1} is unique. □

Proposition 21 (Direct multiplication for order 2)**.** *Let $\lambda_2(x) = (m, u, \varepsilon)$ and $\lambda_2(y) = (n, v, \eta)$. Define the raw product triple by*

$$M = mn, \qquad S = \eta m^2 + \varepsilon n^2 - \varepsilon\eta,$$

$$U = m^2 v + n^2 u + 2uv - u\eta - v\varepsilon - \left\lfloor \frac{S}{2} \right\rfloor, \qquad E = S \bmod 2.$$

Then

$$\rho_2(x)\rho_2(y) = M^2 + 2U - E.$$

Since $\rho_2(x) \geq m^2$ and $\rho_2(y) \geq n^2$, the raw triple cannot lie below layer M. Thus it can fail to be canonical only on the high side, in which case one repeatedly applies the upward carry rule

$$(m, u, \varepsilon) \longmapsto (m+1, u-m-\varepsilon, 1-\varepsilon) \qquad \textit{while } u > m$$

This process terminates at a unique canonical triple (m, u, ε), and the corresponding product word is

$$x \otimes_2 y = \begin{cases} (u, m), & \varepsilon = 0 \textit{ and } u < m, \\ (m, 0), & \varepsilon = 0 \textit{ and } u = m, \\ (m, u), & \varepsilon = 1. \end{cases}$$

Hence order-2 multiplication can also be carried out directly in the onion representation, without global rank inversion.

Proof. Writing $\rho_2(x) = m^2 + 2u - \varepsilon$ and $\rho_2(y) = n^2 + 2v - \eta$, we obtain

$$\rho_2(x)\rho_2(y) = (mn)^2 + 2(m^2 v + n^2 u + 2uv - u\eta - v\varepsilon) - S.$$

Since $S = 2\lfloor S/2 \rfloor + (S \bmod 2)$, this becomes

$$\rho_2(x)\rho_2(y) = M^2 + 2U - E,$$

which is the desired raw product formula. Moreover,

$$\rho_2(x)\rho_2(y) \geq m^2n^2 = M^2,$$

so the represented integer cannot lie below layer M.

The two upward carry rules are the identities

$$m^2 + 2u = (m+1)^2 + 2(u-m) - 1$$

and

$$m^2 + 2u - 1 = (m+1)^2 + 2(u-m-1),$$

so each carry preserves the represented integer. If $u > m$ in the even case then $m^2+2u > (m+1)^2-1$, while if $u > m$ in the odd case then $m^2+2u-1 \geq (m+1)^2$. Hence the canonical representative lies in a larger layer whenever a carry is applied. After a carry, the excess over the top of the layer strictly decreases:

$$(u-m) \longmapsto u - 2m - 1$$

in the even case and

$$(u-m) \longmapsto u - 2m - 2$$

in the odd case. Therefore the process terminates. The final triple is unique because ρ_2^{-1} is unique. □

Corollary 19 gives a closed rank/unrank description, while Proposition 20 and Proposition 21 show that addition and multiplication can be performed directly in the representation by finite layer carries. In the reverse prefer-max order, the layers are therefore not just a coarse size classification: they also provide the normalization mechanism that turns raw algebraic data back into canonical words. The next example illustrates both carry-based viewpoints.

Example 22. In the order-2 reverse prefer-max onion sequence we have

$$\rho_2(24) = 20, \qquad \rho_2(31) = 10.$$

For addition, Proposition 20 gives

$$\lambda_2(24) = (4, 2, 0), \qquad \lambda_2(31) = (3, 1, 1).$$

Therefore the raw sum triple is

$$(M, U, E) = (7, 2 + 1 - 4 \cdot 3, 1) = (7, -9, 1).$$

Applying the downward carry twice gives

$$(7, -9, 1) \longmapsto (6, -3, 0) \longmapsto (5, 3, 1),$$

so

$$24 \oplus_2 31 = 53.$$

Likewise, for multiplication Proposition 21 gives

$$\lambda_2(12) = (2, 1, 0), \qquad \lambda_2(20) = (2, 2, 0).$$

Hence the raw product triple is

$$(M, U, E) = (4, 2^2 \cdot 2 + 2^2 \cdot 1 + 2 \cdot 1 \cdot 2, 0) = (4, 16, 0).$$

Applying the upward carry rules twice gives

$$(4, 16, 0) \longmapsto (5, 12, 1) \longmapsto (6, 6, 0),$$

so

$$12 \otimes_2 20 = 60.$$

In the first computation, the sum is found directly in the representation: the raw triple starts in layer 7 *and the carries move it left until it reaches the canonical odd representative* $(5, 3)$*. In the second, the product starts in layer* 4 *and the carries move it right until it reaches the terminal representative* $(6, 0)$*. Thus, in order* 2*, the onion representation already supports direct addition and multiplication by layer normalization. What remains open is to replace these layer-wise procedures by genuinely local rules, analogous to schoolbook arithmetic, and then to extend them to higher orders.*

3.2 Direct arithmetic in order 3

The order-3 case has the same flavor, but each layer is traversed in three phases rather than by a zig-zag. Fix a layer $m \geq 1$ and write $N = m^3 + t$ with $0 \leq t \leq 3m^2 + 3m$. The residue $\varepsilon = t \bmod 3$ tells us which coordinate carries the layer symbol m, while the quotient $s = \lfloor t/3 \rfloor$ records the remaining two coordinates. Writing $s = vm + w$ with $0 \leq w < m$, the generic part of the layer is traversed in triples

$$(w, v, m), \qquad (v, m, w + 1 \bmod m), \qquad (m, w + 1 \bmod m, v),$$

with the third word interpreted as $(m, 0, v + 1)$ when $w = m - 1$. After these triples, the layer ends with the terminal chain

$$(m, m, m), \qquad (m, m, 0), \qquad (m, 0, 0).$$

The next corollary records the corresponding rank and unrank formulas.

Corollary 23 (Explicit rank and unrank for order 3)**.** *Let* ρ_3 *be the rank map of the reverse prefer-max onion sequence of order* 3*. Then for every* $a, b, c \in \mathbb{N}$ *we have*

$$\rho_3(a, b, c) = \begin{cases} (m+1)^3 - (3 - i), & (a, b, c) = m^{3-i}0^i \text{ for some } i \in \{0, 1, 2\} \text{ and } m \neq 0, \\ c^3 + 3bc + 3a, & a < c \wedge b \leq c, \\ a^3 + 3ac + 3b - 1, & b < a \wedge c \leq a, \\ b^3 + 3ab + 3((c - 1) \bmod b) + 1, & c < b \wedge a \leq b. \end{cases}$$

Conversely, $\rho_3^{-1}(0) = (0,0,0)$. *If* $N \geq 1$, *let*

$$m = \lfloor \sqrt[3]{N} \rfloor, \qquad t = N - m^3, \qquad \varepsilon = t \bmod 3, \qquad s = \left\lfloor \frac{t}{3} \right\rfloor,$$

$$w = s \bmod m, \qquad v = \left\lfloor \frac{s}{m} \right\rfloor, \qquad \delta = (w+1) \bmod m.$$

Then

$$\rho_3^{-1}(N) = \begin{cases} m^{3-i}0^i, & N = (m+1)^3 - (3-i) \text{ for some } i \in \{0,1,2\}, \\ (w,v,m), & \varepsilon = 0, \\ (v,m,\delta), & \varepsilon = 1, \\ (m,0,v+1), & \varepsilon = 2 \text{ and } \delta = 0, \\ (m,\delta,v), & \varepsilon = 2 \text{ and } \delta \neq 0. \end{cases}$$

Consequently, the operations

$$(a,b,c) \oplus_3 (d,e,f) = \rho_3^{-1}\big(\rho_3(a,b,c) + \rho_3(d,e,f)\big)$$

and

$$(a,b,c) \otimes_3 (d,e,f) = \rho_3^{-1}\big(\rho_3(a,b,c)\rho_3(d,e,f)\big)$$

*are explicit addition and multiplication rules on the order-*3 *onion representations.*

Proof. For a fixed layer m, the displayed formulas group the offsets in $[m^3, (m+1)^3)$ into triples according to the residue of $t = N - m^3$ modulo 3. The case $\varepsilon = 0$ lists the words whose last coordinate equals m, namely (w,v,m), in the order determined by the quotient-remainder decomposition $s = vm + w$. The case $\varepsilon = 1$ rotates the same data to the middle coordinate, giving $(v, m, (w+1) \bmod m)$. The case $\varepsilon = 2$ rotates once more to the first coordinate, giving $(m, (w+1) \bmod m, v)$, except when the cyclic increment wraps from $m-1$ to 0, in which case the word is $(m, 0, v+1)$. The final three positions of the layer are exactly (m,m,m), $(m,m,0)$, and $(m,0,0)$. Reading off these offsets gives the formula for ρ_3, and conversely the same decomposition recovers ρ_3^{-1}. The formulas for $\oplus_3$ and $\otimes_3$ are then immediate by transport of ordinary addition and multiplication through ρ_3. □

Corollary 23 still describes arithmetic through the global rank map. As in order 2, the next step is to work with canonical layer-offset coordinates and normalize by finitely many carries.

Definition 24 (Canonical order-3 coordinates)**.** *For a word* x *in the reverse prefer-max onion sequence of order* 3, *define* $\lambda_3(x) = (m,u,\varepsilon)$ *by*

$$\rho_3(x) = m^3 + 3u + \varepsilon,$$

where $\varepsilon \in \{0,1,2\}$ *and the admissible range of* u *is*

$$\begin{cases} 0 \leq u \leq m^2 + m, & \varepsilon = 0, \\ 0 \leq u \leq m^2 + m - 1, & \varepsilon = 1 \text{ or } \varepsilon = 2. \end{cases}$$

Conversely, let $\omega_3(m,u,\varepsilon)$ denote the canonical word associated with such a triple. In the generic cases below, write $u = vm + w$ with $0 \le w < m$:

$$\omega_3(m,u,0) = \begin{cases} (w,v,m), & 0 \le u < m^2+m \text{ and } u = vm+w,\ 0 \le w < m, \\ (m,0,0), & u = m^2+m, \end{cases}$$

$$\omega_3(m,u,1) = \begin{cases} (v,m,(w+1) \bmod m), & 0 \le u < m^2+m-1, \\ (m,m,m), & u = m^2+m-1, \end{cases}$$

and

$$\omega_3(m,u,2) = \begin{cases} (m,0,v+1), & 0 \le u < m^2+m-1 \text{ and } (w+1) \bmod m = 0, \\ (m,(w+1) \bmod m, v), & 0 \le u < m^2+m-1 \text{ and } (w+1) \bmod m \ne 0, \\ (m,m,0), & u = m^2+m-1. \end{cases}$$

Proposition 25 (Direct addition for order 3)**.** *Let $\lambda_3(x) = (m,u,\varepsilon)$ and $\lambda_3(y) = (n,v,\eta)$. Define the raw sum triple by*

$$M = m+n, \qquad U = u+v-mn\cdot M + \left\lfloor \frac{\varepsilon+\eta}{3} \right\rfloor, \qquad E = (\varepsilon+\eta) \bmod 3.$$

Then

$$\rho_3(x) + \rho_3(y) = M^3 + 3U + E.$$

Moreover the raw triple cannot overshoot layer M, so it can fail to be canonical only on the low side. In that case one repeatedly applies the downward carry rules

$$(m,u,0) \longmapsto (m-1, u+m^2-m, 1) \qquad \text{if } u < 0,$$
$$(m,u,1) \longmapsto (m-1, u+m^2-m, 2) \qquad \text{if } u < 0,$$
$$(m,u,2) \longmapsto (m-1, u+m^2-m+1, 0) \qquad \text{if } u < 0.$$

This process terminates at a unique canonical triple (m,u,ε), and the corresponding sum word is

$$x \oplus_3 y = \omega_3(m,u,\varepsilon).$$

Hence order-3 addition can be carried out directly in the onion representation, without global rank inversion.

Proof. From $\rho_3(x) = m^3 + 3u + \varepsilon$ and $\rho_3(y) = n^3 + 3v + \eta$ we get

$$\rho_3(x) + \rho_3(y) = m^3 + n^3 + 3u + 3v + \varepsilon + \eta.$$

Since $m^3 + n^3 = (m+n)^3 - 3mn(m+n)$, this becomes

$$\rho_3(x) + \rho_3(y) = M^3 + 3U + E,$$

with M, U, E as above.

To see that there is no upper overflow, note that

$$\rho_3(x) \le (m+1)^3 - 1, \qquad \rho_3(y) \le (n+1)^3 - 1,$$

so

$$\rho_3(x) + \rho_3(y) \le (m+1)^3 + (n+1)^3 - 2 < (m+n+1)^3.$$

Thus the represented integer lies below layer $M+1$, and only a downward carry can be needed.

The carry rules are the identities

$$m^3 + 3u = (m-1)^3 + 3(u + m^2 - m) + 1,$$

$$m^3 + 3u + 1 = (m-1)^3 + 3(u + m^2 - m) + 2,$$

and

$$m^3 + 3u + 2 = (m-1)^3 + 3(u + m^2 - m + 1).$$

Hence each carry preserves the represented integer. If $u < 0$, then $m^3 + 3u + \varepsilon < m^3$, so the canonical representative must lie in a smaller layer. Each carry lowers the layer by 1, hence the process terminates, and uniqueness follows from the uniqueness of ρ_3^{-1}. □

Proposition 26 (Direct multiplication for order 3)**.** *Let $\lambda_3(x) = (m, u, \varepsilon)$ and $\lambda_3(y) = (n, v, \eta)$. Define*

$$M = mn, \qquad S = m^3\eta + n^3\varepsilon + \varepsilon\eta,$$

$$U = m^3 v + n^3 u + 3uv + u\eta + v\varepsilon + \left\lfloor \frac{S}{3} \right\rfloor, \qquad E = S \bmod 3.$$

Then

$$\rho_3(x)\rho_3(y) = M^3 + 3U + E.$$

Since $\rho_3(x)\rho_3(y) \ge m^3 n^3 = M^3$, the raw triple cannot lie below layer M. Thus it can fail to be canonical only on the high side, in which case one repeatedly applies the upward carry rules

$$(m, u, 0) \longmapsto (m+1, u - m^2 - m - 1, 2) \qquad \text{if } u > m^2 + m,$$

$$(m, u, 1) \longmapsto (m+1, u - m^2 - m, 0) \qquad \text{if } u > m^2 + m - 1,$$

$$(m, u, 2) \longmapsto (m+1, u - m^2 - m, 1) \qquad \text{if } u > m^2 + m - 1.$$

This process terminates at a unique canonical triple (m, u, ε), and the corresponding product word is

$$x \otimes_3 y = \omega_3(m, u, \varepsilon).$$

Hence order-3 multiplication can also be carried out directly in the onion representation, without global rank inversion.

Proof. Expanding the product gives

$$\rho_3(x)\rho_3(y) = (m^3 + 3u + \varepsilon)(n^3 + 3v + \eta)$$
$$= m^3n^3 + 3(m^3v + n^3u + 3uv + u\eta + v\varepsilon) + S.$$

Since $S = 3\lfloor S/3 \rfloor + (S \bmod 3)$, this is exactly

$$\rho_3(x)\rho_3(y) = M^3 + 3U + E.$$

Also,

$$\rho_3(x)\rho_3(y) \geq m^3n^3 = M^3,$$

so the represented integer cannot lie below layer M.

The three upward carry rules are the identities

$$m^3 + 3u = (m+1)^3 + 3(u - m^2 - m - 1) + 2,$$
$$m^3 + 3u + 1 = (m+1)^3 + 3(u - m^2 - m),$$

and

$$m^3 + 3u + 2 = (m+1)^3 + 3(u - m^2 - m) + 1.$$

Hence each carry preserves the represented integer. Whenever one of the displayed inequalities holds, the corresponding value is at least $(m+1)^3$, so the canonical representative must lie in a larger layer. Each carry raises the layer by 1, while the represented integer stays fixed, so after finitely many steps the layer reaches the unique value $\lfloor \sqrt[3]{\rho_3(x)\rho_3(y)} \rfloor$. At that point the triple is canonical, and uniqueness again follows from the uniqueness of ρ_3^{-1}. □

Proposition 27 (Carry complexity in orders 2 and 3)**.** *For $d \in \{2, 3\}$, let ρ_d and λ_d denote the corresponding rank and coordinate maps, and write*

$$\lambda_d(x) = (m, u, \varepsilon), \qquad \lambda_d(y) = (n, v, \eta).$$

Let

$$L_d(N) = \left\lfloor \sqrt[d]{N} \right\rfloor.$$

Then the normalization procedure for addition uses exactly

$$c_d^+(x, y) = m + n - L_d\big(\rho_d(x) + \rho_d(y)\big)$$

carry updates, while the normalization procedure for multiplication uses exactly

$$c_d^\times(x, y) = L_d\big(\rho_d(x)\rho_d(y)\big) - mn$$

carry updates.

In particular,

$$0 \leq c_d^+(x, y) \leq m + n, \qquad 0 \leq c_d^\times(x, y) \leq m + n.$$

Hence, in both orders, direct addition and multiplication require at most $m + n$ carry updates, and therefore only linearly many elementary arithmetic operations in the input layers.

Proof. In Proposition 20 and Proposition 25, the raw sum triple starts in layer $m+n$. Each downward carry preserves the represented integer and lowers the layer by exactly 1. The normalization stops precisely when the triple reaches the canonical layer of $\rho_d(x)+\rho_d(y)$, namely $L_d(\rho_d(x)+\rho_d(y))$. Therefore the number of addition carries is exactly

$$m+n-L_d\big(\rho_d(x)+\rho_d(y)\big).$$

Similarly, in Proposition 21 and Proposition 26, the raw product triple starts in layer mn. Each upward carry preserves the represented integer and raises the layer by exactly 1. The normalization stops when the layer reaches the canonical layer of $\rho_d(x)\rho_d(y)$, namely $L_d(\rho_d(x)\rho_d(y))$. Hence the number of multiplication carries is exactly

$$L_d\big(\rho_d(x)\rho_d(y)\big)-mn.$$

The addition bound is immediate. For multiplication, since

$$\rho_d(x)<(m+1)^d, \qquad \rho_d(y)<(n+1)^d,$$

we have

$$\rho_d(x)\rho_d(y)<((m+1)(n+1))^d.$$

Therefore

$$L_d\big(\rho_d(x)\rho_d(y)\big)<(m+1)(n+1)=mn+m+n+1,$$

so

$$c_d^{\times}(x,y)\le m+n.$$

This proves the claimed bounds. □

Example 28. For layer $m=2$, the formulas give the order

$$\begin{aligned}&002,\ 021,\ 210,\ 102,\ 020,\ 201,\ 012,\ 121,\ 211,\ 112,\\&120,\ 202,\ 022,\ 221,\ 212,\ 122,\ 222,\ 220,\ 200.\end{aligned}$$

For instance,

$$\rho_3(1,2,0)=2^3+3\cdot 1\cdot 2+3((0-1)\bmod 2)+1=18,$$

while

$$\rho_3^{-1}(23)=(1,2,2)$$

because $23=2^3+15$, so $\varepsilon=0$, $s=5$, $v=2$, and $w=1$. For direct addition, the new proposition gives

$$\lambda_3(120)=(2,3,1), \qquad \lambda_3(021)=(2,0,1),$$

$$(M,U,E)=(4,3+0-2\cdot 2\cdot 4,2)=(4,-13,2).$$

A single downward carry sends this to $(3,0,0)$, hence

$$120\oplus_3 021=003.$$

Thus order 3 admits not only explicit rank/unrank formulas, but also a genuine carry-normalization calculus parallel to the order-2 case.

4 Conclusions and future directions

Onion De Bruijn sequences have both a rigid side and a flexible side. Structurally, every onion sequence decomposes into layers indexed by the maximal symbol, and the onion condition forces the boundaries between those layers. Enumeratively, Theorem 9 and Corollary 11 show that each layer admits many Hamiltonian orders and that these choices are independent once the boundary words are fixed. Reverse prefer-max is therefore distinguished not by uniqueness, but by the extra arithmetic regularity of one compatible family of layer orders.

That regularity already yields arithmetic at the structural level. Proposition 12, Corollary 13, and Corollary 14 show that the maximal symbol determines the exact layer of the represented integer, that perfect nth powers are the carry thresholds between layers, and that dominant symbols are monotone in fixed contexts. For the reverse prefer-max sequences of orders 2 and 3, Proposition 17, Corollary 19, Proposition 20, Proposition 21, Corollary 23, Proposition 25, and Proposition 26 sharpen this into explicit rank/unrank formulas and direct carry-normalized arithmetic. Proposition 27 then makes the algorithmic content explicit by giving exact carry counts and linear carry complexity in both orders.

This arithmetic differs from more classical non-standard numeration systems. In mixed-radix systems, including the factorial number system, carries move between predetermined positions with prescribed radices [7]. In word-based systems such as abstract numeration systems and Ostrowski numeration [11, 10], integers are ordered through a language and arithmetic is often studied automata-theoretically. Onion arithmetic combines aspects of both settings, but with a crucial extra constraint: the representing words must appear as consecutive windows of a single infinite De Bruijn sequence. The main carry mechanism therefore moves between layers of a fixed-length window, rather than between positions.

The finite-alphabet literature already supplies much of the relevant toolkit. The Fredricksen–Maiorana theorem identifies the prefer-min De Bruijn sequence with the concatenation, in lexicographic order, of the Lyndon words whose lengths divide n [9]; see also [3] for a shift-rule proof. On the algorithmic side, [1], [3], [2], and [14] provide shift rules, jump rules, and ranking/unranking procedures for fixed alphabets. The next step is to make that toolkit compatible with growing alphabets. In particular, one would like successor and jump rules for the infinite reverse prefer-max order, and higher-order rank/unrank descriptions that expose the correct intra-layer offsets. The central open problem is to understand those offsets well enough to obtain genuinely local addition, multiplication, and eventually division on words of fixed length n.

References

[1] Gal Amram, Yair Ashlagi, Amir Rubin, Yotam Svoray, Moshe Schwartz, and Gera Weiss. An efficient shift rule for the prefer-max de Bruijn sequence. *Discrete Mathematics*, 342(1):226–232, 2019.

[2] Gal Amram and Amir Rubin. An efficient generalized shift-rule for the prefer-max de Bruijn sequence. *Discrete Mathematics*, 343(2):111657, 2020.

[3] Gal Amram, Amir Rubin, Yotam Svoray, and Gera Weiss. De Bruijn sequences: from games to shift-rules to a proof of the Fredricksen–Kessler–Maiorana theorem. *The Electronic Journal of Combinatorics*, 32(3):P3.11, 2025.

[4] Gal Amram, Amir Rubin, and Gera Weiss. A cycle joining construction of the prefer-max de Bruijn sequence, 2021. Preprint, arXiv:2104.02999 [cs.DM].

[5] Verónica Becher and Pablo Ariel Heiber. On extending de Bruijn sequences. *Information Processing Letters*, 111(18):930–932, 2011.

[6] Nicolaas Govert de Bruijn. A combinatorial problem. *Proceedings of the Section of Sciences of the Koninklijke Nederlandse Akademie van Wetenschappen te Amsterdam*, 49(7):758–764, 1946. Also published in Indagationes Mathematicae 8 (1946), 461–467.

[7] Miloš Ercegovac and Tomás Lang. *Digital arithmetic*. Morgan Kaufmann, San Francisco, CA, 2003.

[8] Lester Randolph Ford, Jr. A cyclic arrangement of m-tuples. Technical Report P-1071, RAND Corporation, Santa Monica, CA, April 1957. Report dated April 23, 1957.

[9] Harold Fredricksen and James Maiorana. Necklaces of beads in k colors and k-ary de Bruijn sequences. *Discrete Mathematics*, 23(3):207–210, 1978.

[10] Philipp Hieronymi and Alonza Terry. Ostrowski numeration systems, addition, and finite automata. *Notre Dame Journal of Formal Logic*, 59(2):215–232, 2018.

[11] Pierre Lecomte and Michel Rigo. Abstract numeration systems. In Valérie Berthé and Michel Rigo, editors, *Combinatorics, Automata and Number Theory*, pages 108–162. Cambridge University Press, 2010.

[12] Jonathan Margoliash. Matrix-tree theorem for directed graphs, 2010. University of Chicago VIGRE REU paper.

[13] Monroe Harnish Martin. A problem in arrangements. *Bulletin of the American Mathematical Society*, 40(12):859–864, 1934.

[14] Joe Sawada and Aaron Williams. Practical algorithms to rank necklaces, Lyndon words, and de Bruijn sequences. *Journal of Discrete Algorithms*, 43:95–110, 2017.

[15] William Thomas Tutte. The dissection of equilateral triangles into equilateral triangles. *Mathematical Proceedings of the Cambridge Philosophical Society*, 44(4):463–482, 1948.

[16] William Thomas Tutte and Cedric Austen Bardell Smith. On unicursal paths in a network of degree 4. *The American Mathematical Monthly*, 48(4):233–237, 1941.

[17] Tatyana Pavlovna van Aardenne-Ehrenfest and Nicolaas Govert de Bruijn. Circuits and trees in oriented linear graphs. *Simon Stevin. Wis- en Natuurkundig Tijdschrift*, 28:203–217, 1951.

A Computational verification and software

The code supplementing this paper is available in a companion repository at `https://example.com/onion-de-bruijn`. This appendix summarizes the contents of that repository and the scope of the checks that were carried out.

Available scripts. The file `order2_debruijn_arithmetic.py` implements the order-2 arithmetic developed in the paper: the rank map ρ_2, its inverse, the canonical coordinate map λ_2, the reconstruction map from canonical coordinates, and the direct addition and multiplication procedures obtained by carry normalization. The file `order3_debruijn_arithmetic.py` provides the analogous order-3 tools, including rank/unrank, canonical coordinates, and direct addition and multiplication.

Verification procedure. The file `verify_results.py` is a computational verification script. It generates prefer-max and reverse prefer-max prefixes directly, checks the onion property in small cases, verifies the layer-count and onion-prefix counting formulas on small exhaustively computable instances, and tests the arithmetic formulas by comparing the direct operations with rank arithmetic. In particular, the current script checks the onion theorem on 14 small (n, k) instances, the layer-count formula on 10 small cases, the compatible onion-prefix count on 9 small cases, and universal structural/arithmetic consequences on 2032 exhaustively generated finite onion prefixes. It also verifies order-2 rank/unrank on 500 states and direct addition, multiplication, and carry counts on 14,400 pairs, and order-3 rank/unrank on 900 states together with direct addition, multiplication, and carry counts on 8,100 pairs.

Use by readers. These scripts are meant to provide readers with working arithmetic models of the onion counting system. They can be used to

1. convert between integers and their order-2 or order-3 onion representations,
2. compute canonical layer-offset coordinates,
3. perform direct addition and multiplication inside the representation, and
4. reproduce the finite computational checks by running the verification script.

The verification code is included as a reproducibility aid for the explicit constructions, examples, and finite checks discussed in the paper.

Department of Computer Science, Ben-Gurion University of the Negev
E-mail addresses: `genosard@post.bgu.ac.il`, `ysavorai@post.bgu.ac.il`, `geraw@bgu.ac.il`